

Thiago Belagamba

TraceGuard: plataforma *self-hosted* de observabilidade, alertas e proteção em runtime

Brasil

2026

Thiago Belagamba

**TraceGuard: plataforma *self-hosted* de observabilidade,
alertas e proteção em runtime**

Trabalho de Conclusão de Curso apresentado
ao Curso de Ciência da Computação como
requisito parcial para obtenção do título de
Bacharel.

Nome da Instituição
Curso de Ciência da Computação

Orientador: Nome do Orientador

Brasil
2026

Resumo

Sistemas de *Application Performance Monitoring* (APM) comerciais são caros para pequenas e médias empresas, enquanto stacks *open source* fragmentam traces, logs, *uptime* e segurança em produtos distintos. Este trabalho apresenta o TraceGuard, plataforma *self-hosted* que une telemetria de aplicação, mitigação de fadiga de alertas, monitoramento de *uptime* e um módulo de *Runtime Application Self-Protection* (RASP) no mesmo agente Node.js. O recorte científico é a correlação: o mesmo `traceId` — propagado por `AsyncLocalStorage` e pelo padrão W3C Trace Context — liga latência, bloqueio de *scraper* e alerta acionável. Foram realizados três experimentos reproduzíveis: supressão de alertas por *cooldown*, precisão e *recall* do RASP em *dataset* rotulado e *overhead* de ingestão. O motor de alertas não cria uma notificação por *log*: agrega a janela e suprime rajadas em *cooldown* (50% de supressão na execução de referência). O código inclui testes automatizados, integração contínua e *dashboard* com *drill-down* por *trace*.

Palavras-chave: observabilidade. OpenTelemetry. RASP. fadiga de alertas. APM. *self-hosted*.

Abstract

Commercial Application Performance Monitoring (APM) platforms are expensive for small and medium-sized businesses, while open-source stacks split traces, logs, uptime and security across separate products. This work presents TraceGuard, a self-hosted platform that unifies application telemetry, alert-fatigue mitigation, uptime monitoring and a Runtime Application Self-Protection (RASP) module in the same Node.js agent. The scientific cut is correlation: a single `traceId` — propagated through AsyncLocalStorage and the W3C Trace Context standard — links latency, scraper blocking and an actionable alert. Three reproducible experiments were conducted: cooldown-based alert suppression, RASP precision/recall on a labeled dataset, and ingestion overhead. The alert engine does not emit one notification per log line: it aggregates a sliding window and suppresses bursts under cooldown (50% suppression in the reference run). The artifact includes automated tests, CI and a dashboard with per-trace drill-down.

Keywords: observability. OpenTelemetry. RASP. alert fatigue. APM. self-hosted.

Lista de abreviaturas e siglas

APM	Application Performance Monitoring
CNCF	Cloud Native Computing Foundation
DSR	Design Science Research
F1	Média harmônica de precisão e <i>recall</i>
OTel	OpenTelemetry
OTLP	OpenTelemetry Protocol
PME	Pequena e Média Empresa
RASP	Runtime Application Self-Protection
SRE	Site Reliability Engineering
WAF	Web Application Firewall

Sumário

1	Introdução	6
1.1	Contextualização	6
1.2	Problema de pesquisa	6
1.3	Objetivos	6
1.4	Justificativa	7
1.5	Delimitação	7
2	Fundamentação teórica	8
2.1	Observabilidade e <i>tracing</i> distribuído	8
2.2	Fadiga de alertas	8
2.3	RASP versus WAF	8
2.4	Arquitetura de ingestão	8
3	Trabalhos relacionados	9
4	Metodologia	10
5	O artefato	11
5.1	Arquitetura	11
5.2	SDK	11
5.3	Alertas	11
5.4	Segurança da API	12
6	Experimentos e resultados	13
6.1	Fadiga de alertas	13
6.2	Classificação RASP	13
6.3	<i>Overhead</i>	14
7	Discussão	15
8	Limitações e trabalhos futuros	16
9	Conclusão	17
	Referências	18

Lista de tabelas

Tabela 1	– Sistemas relacionados e lacunas no recorte deste trabalho	9
Tabela 2	– Experimento 1 — fadiga de alertas	13
Tabela 3	– Experimento 2 — casos rotulados do RASP	14
Tabela 4	– Experimento 2 — métricas de classificação	14
Tabela 5	– Experimento 3 — latência (80 amostras)	14

1 Introdução

1.1 Contextualização

Observabilidade é a capacidade de inferir o estado interno de um sistema a partir de saídas externas — classicamente *logs*, métricas e *traces* (Cloud Native Computing Foundation, 2023). No mercado, plataformas SaaS (Datadog, New Relic, Dynatrace) cobram por *host*, gigabyte ou *span*. Para uma PME que precisa manter dados em infraestrutura própria, a alternativa típica é montar Jaeger, Prometheus, Grafana, Uptime Kuma e um WAF. Cada peça resolve um recorte; nenhuma entrega, no mesmo identificador, a história “esta requisição foi lenta, bloqueada como *bot* e gerou um alerta”.

1.2 Problema de pesquisa

Como reduzir o ruído de alertas e correlacionar telemetria de desempenho com eventos de proteção em *runtime*, em uma plataforma *self-hosted* adequada a PMEs, sem exigir um coletor OpenTelemetry completo?

1.3 Objetivos

Objetivo geral: projetar e avaliar uma plataforma unificada de observabilidade e proteção em *runtime* para aplicações Node.js.

Objetivos específicos:

1. Instrumentar *traces* com `AsyncLocalStorage` e cabeçalho W3C `traceparent`.
2. Persistir telemetria de forma assíncrona (HTTP → RabbitMQ → PostgreSQL) e exibi-la em tempo real.
3. Mitigar fadiga de alertas com regras, janela deslizante e *cooldown*, medindo taxa de supressão.
4. Detectar e, opcionalmente, bloquear *scrapers* no próprio processo da aplicação (RASP), correlacionando o evento ao `traceId`.
5. Avaliar o artefato com experimentos reproduzíveis de fadiga, classificação RASP e *overhead*.

1.4 Justificativa

A literatura de SRE trata fadiga de alertas como risco operacional: o excesso de notificações leva o operador a ignorá-las (BEYER et al., 2016). RASP comercial opera dentro do processo e vê o que um WAF na borda não vê (Imperva, 2019). Unir os dois recortes em um agente leve é relevante para PMEs e é demonstrável em banca: um clique no *dashboard* mostra APM e bloqueio no mesmo identificador.

1.5 Delimitação

O trabalho não substitui um *backend* OpenTelemetry (SigNoz, Jaeger, Grafana Tempo). Não implementa *multi-tenant*, operador Kubernetes nem inspeção de SQL no *driver*. O RASP desta prova de conceito é de borda comportamental (taxa, *User-Agent*, varredura, cabeçalhos), não um motor contra o OWASP Top 10 completo.

2 Fundamentação teórica

2.1 Observabilidade e *tracing* distribuído

Um *trace* agrupa *spans* de uma requisição através de serviços. O padrão W3C Trace Context define o cabeçalho `traceparent` no formato `00-{trace-id}-{parent-id}-{flags}` (World Wide Web Consortium, 2021). O OpenTelemetry tornou-se o padrão de fato de instrumentação: os SDKs emitem OTLP e o *backend* é plugável (OpenTelemetry Authors, 2026; Jaeger Authors, 2026; SigNoz, 2026).

O Node.js oferece `AsyncLocalStorage` sobre `async_hooks`, a mesma família de APIs usada pelo SDK JavaScript do OpenTelemetry para propagar contexto sem passar o objeto manualmente em cada *callback* (Node.js Contributors, 2026).

2.2 Fadiga de alertas

Alertar cada evento `level=error` produz N notificações para N *logs*. Práticas de SRE recomendam alertar sintomas (taxa de erro, SLO) com *cooldown*, agrupamento e severidade (BEYER et al., 2016). A métrica deste trabalho é a **taxa de supressão**: fração de disparos persistidos com `suppressed=true` após o *cooldown*.

2.3 RASP versus WAF

WAF inspeciona o tráfego HTTP na borda. RASP executa no processo da aplicação e pode bloquear com contexto de *runtime* (Imperva, 2019). Avaliações acadêmicas de ferramentas de proteção usam matriz de confusão — precisão, *recall*, F1 e índice de Youden (TPR – FPR), no espírito do OWASP Benchmark (OWASP Foundation, 2024). Este TCC aplica essas métricas a um *dataset* rotulado de *scraping*, não ao Benchmark Java completo — limitação declarada.

2.4 Arquitetura de ingestão

Filas (RabbitMQ) desacoplam o aceite HTTP (202 Accepted) da persistência. *Views* materializadas aceleram agregações. WebSocket reduz a latência do *dashboard* em relação a *polling*. Redis pub/sub sincroniza múltiplas réplicas do *hub* em tempo real.

3 Trabalhos relacionados

O Quadro 1 posiciona o TraceGuard diante de sistemas estabelecidos. O recorte não é “substituir o SigNoz”; é unificar APM, RASP e alerta no mesmo `traceId`, com *deploy self-hosted* leve.

Tabela 1 – Sistemas relacionados e lacunas no recorte deste trabalho

Sistema	O que cobre	Lacuna no recorte TraceGuard
SigNoz	APM nativo de OTel, <i>logs</i> , métricas, <i>self-hosted</i>	Sem RASP no agente; stack ClickHouse mais pesada
Jaeger	<i>Tracing</i> CNCF	Sem <i>logs</i> , alertas de negócio, <i>uptime</i> ou RASP
Grafana LGTM	<i>Logs</i> , métricas, <i>traces</i>	Três <i>backends</i> ; sem proteção <i>in-process</i>
Sentry	Erros, performance, <i>releases</i>	RASP e <i>uptime</i> não são o núcleo
Uptime Kuma	<i>Probes</i> HTTP	Sem APM nem correlação com <i>traces</i>
ModSecurity / CrowdSec	WAF / reputação de IP	Fora do processo; sem <code>traceId</code> da aplicação
Datadog / New Relic	Plataforma completa, SaaS	Custo e residência de dados

O TraceGuard compete no recorte “um `docker compose`, um SDK, um *dashboard*, correlação APM+RASP+alerta” (SigNoz, 2026; Jaeger Authors, 2026).

4 Metodologia

Adota-se *Design Science Research* (PEFFERS et al., 2007): identificar o problema, projetar o artefato, demonstrar, avaliar e comunicar.

1. **Artefato:** monorepo TypeScript (`@traceguard/sdk`, *backend* Fastify, *dashboard* Next.js).
2. **Decisões:** registradas em ADRs no diretório `docs/adr/` do repositório.
3. **Avaliação:** três experimentos controlados (Capítulo 6), testes unitários (`pnpm test`) e demonstração no *dashboard*.
4. **Reprodutibilidade:** scripts em `scripts/experiments/` e carga k6 em `scripts/load-test/`.

Não houve estudo com usuários reais de PME; a validade é de laboratório. Isso é uma limitação, não um defeito oculto.

5 O artefato

5.1 Arquitetura

A Figura conceitual do fluxo é a seguinte: a aplicação monitorada carrega o SDK; o SDK envia lotes a `POST /api/v1/ingest`; o Fastify publica no RabbitMQ e responde 202; o *consumer* persiste no PostgreSQL e dispara o EventHub (WebSocket). O *middleware* RASP envia eventos a `/api/v1/ingest/rasp` com o mesmo `traceId`. *Probes* de *uptime* no *backend* alimentam alertas e *webhook*.

Listing 5.1 – Fluxo de ingestao e correlacao

Aplicacao + SDK	--POST /api/v1/ingest-->	Fastify	-->	RabbitMQ
		API key opc.		
		WebSocket		v
Dashboard Next.js	<----	EventHub	----	consumer --> PostgreSQL
RASP middleware	--POST /ingest/rasp-->	Fastify (mesmo traceId)		
Uptime probes	----->	alertas + webhook		

5.2 SDK

O agente usa `AsyncLocalStorage` para identificar o *trace* e o *span* (Node.js Contributors, 2026). A função `continueOrCreateContext` lê o cabeçalho W3C `traceparent` ou os cabeçalhos legados `X-Trace-Id` (World Wide Web Consortium, 2021). Requisições HTTP de saída geram um *span* filho e reinserem `traceparent`.

O RASP soma pontuações: `bot_user_agent` 50, `missing_headers` 20, `rate_limit` 40, `sequential_scan` 60. O bloqueio ocorre se a soma for maior ou igual a 70 no modo `block`. Há *whitelist* de *crawlers* (Googlebot, Bingbot e equivalentes), porque o padrão `bot` no *User-Agent* pegaria o Googlebot.

5.3 Alertas

Tipos implementados: `error_rate`, `error_count`, `error_burst`, `rasp_threat_count` e `uptime_down`. O *cooldown* é por par `ruleId:service`. A API expõe `suppressionRate` em 0–100.

5.4 Segurança da API

A variável `API_KEY` é opcional. Sem chave (desenvolvimento), a API permanece aberta. Com chave, ingestão, leitura e WebSocket exigem `x-api-key` ou `?token=`. O *dashboard* envia `NEXT_PUBLIC_API_KEY` e pode exigir senha (`NEXT_PUBLIC_DASHBOARD_PASSWORD`).

6 Experimentos e resultados

Os experimentos foram executados em 19 de setembro de 2026, em máquina local, com *backend*, PostgreSQL e RabbitMQ. Os protocolos estão em `scripts/experiments/`. Recomenda-se registrar CPU, memória e sistema operacional na versão impressa.

6.1 Fadiga de alertas

Hipótese: uma rajada de erros não gera uma notificação por evento; uma segunda rajada em *cooldown* é suprimida.

A linha de base ingênua considera 150 notificações (um alerta por evento de erro). O Quadro 2 mostra a execução de referência (`pnpm experiment:fatigue`).

Tabela 2 – Experimento 1 — fadiga de alertas

Etapa	Acionáveis	Suprimidos	Taxa
Antes	0	0	0%
Após 100 erros	3	0	0%
Após +50 erros (<i>cooldown</i>)	3	3	50%

O motor agrega a janela. A primeira rajada dispara as três regras padrão (`error_rate`, `error_count`, `error_burst`). A segunda, ainda em *cooldown*, é persistida como suprimida. Delta da execução: 3 acionáveis e 3 suprimidos (50%). O sistema nunca cria um alerta por linha de *log*.

6.2 Classificação RASP

Hipótese: *scrapers* com *User-Agent* conhecido em rotas `/api/*` são bloqueados; *browser*, *healthcheck* e Googlebot não.

O *script* `pnpm experiment:rasp` usa `http.request` (não o `fetch` do Node.js), porque o `fetch` injeta `Accept-Language: *` e o *scraper* deixaria de somar `missing_headers` (pontuação $50 < 70$). O *rate limiter* foi zerado (API recém-iniciada). Os Quadros 3 e 4 resumem o resultado oficial do autor.

O *dataset* é pequeno e controlado: adequado a laboratório, insuficiente para afirmar “RASP 100% em produção”.

Tabela 3 – Experimento 2 — casos rotulados do RASP

Caso	Esperado	Detectado
browser_legitimo	legítimo	permitido
healthcheck_curl	legítimo	permitido
scraper_python	ameaça	bloqueado
scraper_scrapy	ameaça	bloqueado
crawler_googlebot	legítimo	permitido

Tabela 4 – Experimento 2 — métricas de classificação

Métrica	Valor
Verdadeiros positivos (TP)	2
Falsos positivos (FP)	0
Falsos negativos (FN)	0
Verdadeiros negativos (TN)	3
Precisão	1,000
<i>Recall</i>	1,000
F1	1,000
Youden	1,000

6.3 *Overhead*

Hipótese: o percentil 95 da ingestão fica abaixo de 500 ms em desenvolvimento, alinhado ao limiar do k6.

O comando `pnpm experiment:overhead` coletou 80 amostras (Quadro 5).

Tabela 5 – Experimento 3 — latência (80 amostras)

Alvo	Média (ms)	p50 (ms)	p95 (ms)
GET /health (RASP analisa)	1,6	1,5	2,1
POST /api/v1/ingest	2,0	1,8	3,2

O p95 da ingestão (3,2 ms) está muito abaixo do alvo de 500 ms.

7 Discussão

O diferencial não é “mais um *dashboard*”. É a correlação demonstrável e a métrica de fadiga. Chamar o módulo de RASP exige honestidade: trata-se de proteção contra *scraping*, não de substituto de WAF contra injeção. As heurísticas em `fingerprints.ts` são simples — o texto admite isso e publica FP/FN em vez de invocar aprendizado de máquina.

O monitor de *event loop* usa `monitorEventLoopDelay` (não eventos de *garbage collection*). *Spans* filhos existem nas saídas HTTP. Testes automatizados cobrem o núcleo que a arguição pode abrir no projetor.

8 Limitações e trabalhos futuros

- O RASP do SDK atende apenas Express; Fastify fica como trabalho futuro.
- *Cooldown* e EventHub sem Redis perdem estado no reinício ou entre réplicas (Redis é opcional).
- Não há exportador OTLP: há acoplamento ao *backend* TraceGuard.
- Não houve estudo de campo com operadores de PME.
- Autenticação do *dashboard* com variáveis `NEXT_PUBLIC_*` expõe a chave no navegador — aceitável na prova de conceito, insuficiente para produção.

Como trabalhos futuros propõem-se exportador OTLP, retenção com TTL, *dead-letter* no RabbitMQ, *adapter* Fastify e *dataset* RASP maior.

9 Conclusão

O TraceGuard responde à pergunta de pesquisa com um artefato executável: agente Node.js, *pipeline* assíncrono, *dashboard* em tempo real, motor de fadiga e RASP correlacionado. Os experimentos mostram que o sistema não alerta um-para-um e que as heurísticas de *scraping* são mensuráveis. O trabalho é de engenharia de software com avaliação empírica de laboratório — adequado a TCC, desde que o texto e a demonstração de oito minutos sejam o centro da defesa, não a quantidade de abas da interface.

Referências

BEYER, B. et al. **Site Reliability Engineering: How Google Runs Production Systems**. Sebastopol: O'Reilly Media, 2016.

Cloud Native Computing Foundation. **Cloud Native Observability Whitepaper**. 2023. Disponível em: <<https://www.cncf.io>>.

Imperva. **A Guide to Runtime Application Self-Protection (RASP)**. [S.l.], 2019. Disponível em: <<https://www.imperva.com/resources/whitepapers/Imperva-A-Guide-to-RASP.pdf>>.

Jaeger Authors. **Jaeger: open source distributed tracing**. 2026. Disponível em: <<https://www.jaegertracing.io>>.

Node.js Contributors. **AsyncLocalStorage — Node.js documentation**. 2026. Disponível em: <https://nodejs.org/api/async_context.html>.

OpenTelemetry Authors. **OpenTelemetry Documentation**. 2026. Disponível em: <<https://opentelemetry.io>>.

OWASP Foundation. **OWASP Benchmark Project**. 2024. Disponível em: <<https://owasp.org/www-project-benchmark/>>.

PEFFERS, K. et al. A design science research methodology for information systems research. **Journal of Management Information Systems**, v. 24, n. 3, p. 45–77, 2007.

SigNoz. **SigNoz: open-source OpenTelemetry-native observability**. 2026. Disponível em: <<https://signoz.io>>.

World Wide Web Consortium. **Trace Context**. [S.l.], 2021. Disponível em: <<https://www.w3.org/TR/trace-context/>>.